

Foundations of Reinforcement Learning and Control: Connections and New Perspectives

Claire Vernade^{1,*}, Onno Eberhard^{2,*}, Martha White³, Florian Dörfler⁴, Csaba Szepesvári³,
Miroslav Krstic⁵, and Michael Muehlebach²

¹University of Technology Nuremberg, claire.vernade@utn.de

²Max Planck Institute for Intelligent Systems, Tübingen, Germany, oeberhard@tue.mpg.de,
michaelm@tue.mpg.de

³University of Alberta and Alberta Machine Intelligence Institute (Amii), whitem@ualberta.ca,
szepesva@ualberta.ca

⁴ETH Zürich

⁵University of California San Diego

*Equal contribution.

Abstract

Reinforcement learning and control theory are two adjacent scientific fields that focus on optimizing the controller of unknown dynamical systems using feedback. While both fields have common roots in dynamic programming, they have evolved with distinct methodologies, goals, and cultures. Despite decades of mutual influence, a significant gap persists between the two communities. This tutorial introduces adaptive control, actor-critic reinforcement algorithms, and a new way to combine these two paradigms for data-driven decision making on a classical locomotion control problem. Our aim is to provide a foundation for understanding the core differences between the two approaches and insights to help experts in each field better understand and engage with the tools and approaches of the other.

Keywords: reinforcement learning; control theory; adaptive control; dynamic programming; actor-critic methods

1 Introduction

Dynamical systems are ubiquitous in physical, biological and digital environments, and controlling them towards given targets has been an ongoing challenge for more than a century. Control theory and reinforcement learning (RL) are now at the forefront of this challenge, and are active fields of research tackling data-driven decision making. Despite fundamentally different histories, the similarity of the challenges has recently pushed the convergence of the two communities, emphasizing adaptation, universality and broad applicability of the produced methods.

Adaptive control emerged in the 1960s as a powerful response to a central engineering challenge: how to maintain stability and performance when the dynamics of a system are uncertain or change over time [6, 7]. Motivated by practical needs in aerospace, manufacturing, and process control, the field quickly gained prominence. Early experiments, such as gain-scheduled controllers for high-performance aircraft [64] and self-tuning regulators for industrial plants [16, 31], demonstrated that it was possible to design feedback systems that could adjust in real time to uncertain or evolving dynamics. These successes generated much optimism: adaptive systems appeared to behave “intelligently” by tuning themselves in real time, offering promising performance in uncertain and time-varying environments. The theoretical foundations developed in this period, including model-reference adaptive control (MRAC) [42, 68] and adaptive pole placement, laid the groundwork for decades of research.

With the start of the 1990s, the attention in adaptive control turned from linear systems to nonlinear systems. Fuelled by the structural advances in geometric nonlinear control, the highly cited adaptive backstepping [52] provided parameter-adaptive Lyapunov-based designs for feedback-linearizable systems, as well as systems beyond this class, with unlimited amounts of parametric uncertainty. The application range of adaptive nonlinear control is broad: robotics, vehicles, electric machines, power electronics, propulsion and flows, and numerous other applications.

In parallel, in the 1980s, neural networks [77] emerged in artificial intelligence as complex families of parametrized functions that could be tuned using data, provided that the optimization could be performed in an efficient way. This learning mechanism can also be viewed as a dynamical system, where current parameter values are updated sequentially using available data. The similarity with brain development prompted cognitive scientists to search for optimization algorithms that would resemble the processes occurring in the human brain. One of the key mechanisms at stake in this natural optimization process is the reinforcement of connections through the backward propagation of dopamine signals. This idea inspired the *temporal difference* algorithm [86], originally aimed at optimizing neural networks but soon connected to adaptive control [90] and approximate dynamic programming [14], giving rise to the research field now known as *reinforcement learning*.

In recent years, both RL and control have evolved due to advances in data availability and computational infrastructure. This new technological environment has led both fields to converge. A growing community of control researchers now approach control problems from a data-centric, algorithmic, and computational perspective [22]. In parallel, RL researchers are now increasingly working on physical systems, aiming at exploiting domain knowledge and providing stronger guarantees for downstream users [24, 101].

These developments have enabled the study and deployment of control strategies in complex, high-dimensional environments, where challenges go well beyond classical notions of regulation or stabilization [29, 39, 61, 62, 75, 91]. Modern applications demand controllers capable of handling non-equilibrium motion, non-smooth dynamics, and changing goals in real time. Such demands are increasingly common in domains like robotic locomotion, manipulation, and interactive decision making—areas that continue to push the limits of both learning and control.

Recent progress points to a renewed opportunity to revisit and significantly expand the core ambitions of adaptation and learning set out by adaptive control. With the convergence of data-driven algorithms, robust control theory, and scalable computation, the time is ripe to bring together the RL and control communities. This tutorial contributes to that effort by clarifying how each field conceptualizes the problem of learning to control unknown systems, identifying their complementary strengths, and outlining a path toward more unified and practically effective approaches.

Structure of the tutorial.

We aim at providing a concise but technical introduction to both research fields to allow newcomers as well as researchers from either side to join efforts.

The tutorial starts with a general introduction (Section 2) to the learning problem, introducing a unified notation and emphasizing the common principles that underlie both RL and control, namely dynamical systems, Lyapunov functions and dynamic programming. We introduce a locomotion task as a running example to illustrate the methods throughout the tutorial. Additionally, we discuss in more detail the methodological differences between the fields and how they form two avenues toward the common problem of *data-driven decision making*.

The core technical section of this tutorial then explains in detail two main approaches that we chose as representative methodologies from the two fields. We first discuss *model-reference adaptive control* (MRAC, Sec. 3.1, with extensions in Sec. 3.2), where data-driven adaptivity is added on top of a base model of the system. In contrast, we introduce *actor-critic* algorithms (Sec. 3.3) as major representatives of model-free reinforcement learning. We then present an original and successful combination of these two approaches, using an actor-critic algorithm for high-level learning and planning and MRAC for low-level adaptation (Sec. 3.4). We also discuss the distinction between model-based and model-free algorithms in RL and blur the standard boundaries to further highlight connections with control.

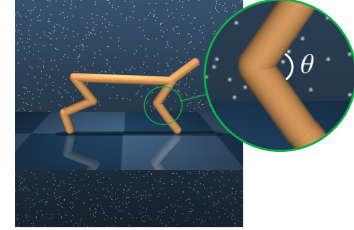
2 Problem definition

At their heart, both RL and control theory are concerned with the general problem of sequentially selecting actions to make a dynamical system achieve a desired behavior. In this section, we formalize the problem setting we consider in this article, and clarify the different terminology used by control theorists and reinforcement learners. We begin by introducing the *Half-Cheetah* system, a simulated locomotion task, which will be a running example for the rest of the tutorial. We then present the foundations of RL and adaptive control, starting with a discussion of the common foundation on which both fields are based: dynamical systems and optimal control.

2.1 Running example: the Half-Cheetah system

The Half-Cheetah, shown on the right, is a simplified robotic system simulated in MuJoCo [95].¹ This system is the basis of a common benchmark task for RL algorithms, with the goal of making the robot ‘run’ as far as possible within a given time limit.

In Section 3.3, we illustrate how a modern deep RL algorithm, *Soft Actor-Critic* (SAC), can learn a locomotion policy directly from interaction with the environment. In this setting, the objective is to maximize performance—for example, making the robot run as fast as possible—without imposing a reference trajectory that the robot must follow.



By contrast, robotic systems such as the one considered here have long been studied in control theory, where the objective is often formulated in terms of stability or trajectory tracking. For example, the Half-Cheetah system can be decomposed into six individual joints whose angles θ are used to describe the system. Though the governing equations are complex and highly nonlinear, the behavior of these joints can be approximately described by independent second-order linear dynamical systems:

$$\ddot{\theta} + 2\zeta\omega\dot{\theta} + \omega^2\theta = \frac{\tau}{I} + \eta, \quad (1)$$

where ζ is a damping (friction) coefficient, ω is the natural angular frequency of the joint, $\frac{\tau}{I}$ is the applied torque divided by the rotational inertia, and η is a “disturbance” term that captures all nonlinear effects. This is a useful model if η is small, and in Section 3.1, we show how MRAC enables reference tracking in such a second-order system, even if model parameters are unknown.

RL and control theory traditionally see control problems through different lenses, and in this article we argue that these different points of view are often complementary. As an illustration, we develop an original method for combining high-level RL with low-level MRAC. In Section 3.4, we test this approach in the Half-Cheetah environment, and show that it enables both high performance (thanks to the actor-critic algorithm) and continual adaptation to changing low-level dynamics (thanks to the adaptive controller).

Beyond the Half-Cheetah benchmark, similar architectures are natural whenever a complex system admits a useful separation between high-level decision making and low-level stabilization. Examples include legged robots operating on changing terrain, aerial or underwater vehicles with uncertain payloads or fluid interactions, robotic manipulators whose dynamics change with the grasped object, assistive devices such as prostheses or exoskeletons that must adapt to the user, and chemical processing plants that operate under varying loads and feedstock compositions. In such settings, RL can be used to learn task-level behavior from experience, while MRAC can provide continual adaptation of the low-level tracking controllers.

¹We use the DeepMind Control Suite implementation [92, 102]. The image of the Cheetah has been reproduced with permission from [92].

2.2 Preliminaries: dynamical systems

RL and control theory have developed independently, and this is reflected in the notation and assumptions commonly used, even in their treatment of the fundamental object underlying both fields: the dynamical system. We provide a unified language and notation (summarized in Appendix A) and highlight that RL and control are fundamentally concerned with the same problem: designing a controller, or *policy*, that drives a dynamical system towards a goal.

A dynamical system is a model mainly composed of three elements:

- A state space $\mathcal{S}$: this is a set of variables that describes the world the policy, or controller, is interacting with;
- An action space $\mathcal{A}$: these are the controls that are available to the policy to impact the environment, i.e., modify the state;
- A dynamics function that dictates how the state changes as a function of the current state and the action chosen by the learner. The dynamics may or may not depend on time, and the two fields have adopted different notations (see box below).

In RL, the standard modeling formalism is a *controlled Markov chain*, or Markov decision process, induced by a kernel that defines state transitions conditionally on actions. In control, it is more common to model the dynamics of a physical system with a deterministic function and an additive noise term representing external perturbations. This can be summarized as follows:

RL view.

RL adopts a stochastic formulation based on controlled Markov chains. At round t , given a state $s_t \in \mathcal{S}$ and an action $a_t \in \mathcal{A}$, the next state evolves as

$$s_{t+1} \sim p(\cdot \mid s_t, a_t),$$

where $p(\cdot \mid s, a)$ is the distribution of the next state.

Control view.

Control is typically formulated for physical systems with deterministic dynamics subject to disturbances. The state evolution is written

$$s_{t+1} = f(s_t, a_t) + \eta_t,$$

where f is a (possibly known) map and η_t represents external perturbations.

While the underlying problem is shared, the two fields differ in their formulations and solution methods. A central aim of this tutorial is to bridge these perspectives. We therefore begin by introducing the key concepts, objectives, and tools used in each community.

2.3 Stability and robustness

A central objective in control is to design policies that ensure that the dynamical system behaves in a predictable and safe manner over time. A natural first approach is to input a fixed sequence of actions into the system independently of observed states; this is called *open-loop control*. However, to achieve stability, a feedback policy is often needed: given a target state $s^* \in \mathcal{S}$, one seeks a policy π that takes an observation of the state as input (called *closed-loop control*) such that the resulting trajectory $(s_t)_{t \geq 0}$ remains close to s^* , or converges to it. In the deterministic setting, this amounts to requiring that s^* is an equilibrium of the closed-loop system,

$$f(s^*, \pi(s^*)) = s^*,$$

and that trajectories initialized near s^* remain close (stability) or converge to it (asymptotic stability). In the presence of stochastic perturbations, these requirements are relaxed to hold in expectation or with high probability, leading to notions of robustness with respect to disturbances.

A standard tool to analyze such stability properties is the notion of a *Lyapunov function*. A function $U : \mathcal{S} \rightarrow \mathbb{R}_+$ is called a Lyapunov function for a policy π around $s^\star$ if

$$\begin{aligned} U(s^\star) &= 0, \quad U(s) > 0 \quad \text{for all } s \neq s^\star, \\ U(f(s, \pi(s))) - U(s) &\leq 0 \quad \text{for all } s \in \mathcal{S}. \end{aligned}$$

The function U can be interpreted as a measure of distance or energy with respect to the target state, and the second condition enforces that this quantity decreases along trajectories. Stronger conditions, such as a strict decrease away from $s^\star$, imply asymptotic convergence to the target state.

Lyapunov functions provide a powerful and general framework. However, they are not given a priori. Constructing a suitable function U is hard work, demanding skill and experience, accumulated over a large body of literature [48]. Since the work of Krstić et al. [52], much of the adaptive control literature—comprising thousands of papers over the past three decades—has focused not only on the design of adaptive controllers, but more fundamentally on the construction of Lyapunov functions guaranteeing stability.

2.4 Optimal control & decision-making

The linear-quadratic control approach: a first value function. While there is no general recipe for arbitrary systems, the case of linear dynamical systems is well understood and provides a pedagogical example of a situation where Lyapunov functions can be obtained through an optimal control objective. Consider a linear dynamical system

$$s_{t+1} = As_t + Ba_t,$$

where the target is $s^\star = 0$, and A and B are given.² We aim to construct a linear policy $a_t = Ks_t$, which simplifies the closed-loop dynamics into $s_{t+1} = (A + BK)s_t$. A natural candidate Lyapunov function U is quadratic: for some choice of $P \succ 0$,

$$U(s) = s^\top Ps.$$

Certifying stability for our controller K then boils down to ensuring that

$$(A + BK)^\top P(A + BK) - P \preceq 0.$$

This reduces the problem of verifying stability to checking a matrix inequality, and forms the basis of much of linear control theory [15].

However, rather than constructing such a Lyapunov function directly, an alternative perspective is to define an optimization objective and derive both the policy and the Lyapunov function from it. This is the viewpoint adopted in the linear quadratic regulator (LQR) formulation, where one considers the cost

$$\sum_{t=0}^{\infty} (s_t^\top Qs_t + a_t^\top Ra_t),$$

for some known matrices $Q \succeq 0$ and $R \succ 0$ chosen by the designer. These matrices typically encode a trade-off between penalizing deviations of the state from the target and limiting the magnitude of the control actions. The goal is then to find a policy minimizing this cumulative cost.

More generally, given a sequence of actions $(a_t)_{t \geq 0}$ output by a controller π , the *cost-to-go function* (or negative *value function*) associated with this objective is defined as the expected cumulative cost when starting from a state s :

$$J^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} (s_t^\top Qs_t + a_t^\top Ra_t) \mid s_0 = s \right],$$

²In adaptive control, a prescribed pair of matrices can instead define the reference model that guides adaptation based on feedback (see Section 3.1).

where the expectation is taken over the trajectories induced by the dynamics and the policy. The value function therefore quantifies the long-term performance of a policy from any initial state.

In this setting, it can be shown, e.g., by dynamic programming (explained further below), that the optimal policy is linear, and the associated optimal cost-to-go takes the quadratic form

$$J^*(s) = s^\top P^* s,$$

where P^* is the solution of the algebraic Riccati equation [4, 13].

Cost-to-go functions and Lyapunov functions originate from different perspectives. The former quantify the long-term performance with respect to a given objective while the latter certify stability of an equilibrium. However, for LQR (and beyond, as we shall see soon), a remarkable alignment occurs: the optimal cost-to-go function J^* is also a Lyapunov function for the closed-loop system. Indeed, along trajectories of the optimal policy,

$$J^*(s_{t+1}) - J^*(s_t) = -(s_t^\top Q s_t + a_t^\top R a_t) \leq 0,$$

so that the decrease of the cost-to-go exactly matches the instantaneous cost. As a consequence, optimality and stability are achieved simultaneously.

This example illustrates a broader perspective: Lyapunov functions provide a direct way to enforce stability through local descent conditions. This viewpoint naturally connects to dynamic programming and RL, which we discuss next.

Markov Decision Processes and Dynamic Programming.

To simplify the exposition, we focus here on discounted infinite-horizon problems. Given a discount factor $\gamma \in [0, 1)$, the goal is to find a feedback controller, or *policy*, $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ that minimizes the expected cumulative cost-to-go function, where $\Delta(\mathcal{A})$ denotes the set of probability distributions over actions:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t c(s_t, a_t) \right] = -\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] = -V(\pi), \quad (2)$$

where the expectation is taken over the system dynamics, the possible stochasticity of the policy, and the initial condition $s_0 \sim p_0$. Here, we have introduced the RL terminology of *rewards* r (negative costs), and *value functions* V (reward-to-go).

RL view. The objective in RL is to maximize cumulative rewards along a trajectory. When rewards are defined as negative costs, $r(s, a) = -c(s, a)$, the reward-to-go is called the *value function* and must be maximized.

Control view. In control, the objective is most often to reach a zero-cost state, which corresponds to the target. Thus, costs are often assumed to be positive, and J must be minimized by the policy.

In this tutorial, we adopt the RL terminology and focus on maximizing value functions.

How can we efficiently compute an optimal policy, if the action taken in a state s affects not only the immediate reward, but also all future states visited by the system? The answer to this computational question is given by *dynamic programming* [12], which relies on the recursive structure of the value functions associated with our Markov decision process. Given a policy π and a state $s \in \mathcal{S}$, the value function of π is defined as the expected downstream rewards when starting in state s and following π :

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_t \sim \pi(\cdot | s_t) \right]. \quad (3)$$

In particular, we have $J(\pi) = -\mathbb{E}_{s \sim p_0} [V^\pi(s)]$. Thus, optimizing J amounts to finding a policy whose value function is maximal on the relevant initial states.

By exploiting the Markov property of the dynamics, one obtains the *Bellman equation* for a fixed policy, which connects the value in a given state s with that of the next reachable ones:

$$V^\pi(s) = \int_{\mathcal{A}} \pi(da|s) \left[r(s, a) + \gamma \int_{\mathcal{S}} V^\pi(s') p(ds'|s, a) \right]. \quad (4)$$

Here, p denotes the transition kernel of the controlled Markov process. In the deterministic case where π maps each state to a single action, this simplifies to $V^\pi(s) = r(s, \pi(s)) + \gamma \int_{\mathcal{S}} V^\pi(s') p(ds'|s, \pi(s))$. To emphasize the impact of a specific action a in a given state s , it is also useful to define the state-action value function, or *Q-function*, associated with π :

$$Q^\pi(s, a) = r(s, a) + \gamma \int_{\mathcal{S}} V^\pi(s') p(ds'|s, a). \quad (5)$$

The value function can then be recovered by averaging the *Q-function* under the policy: $V^\pi(s) = \int_{\mathcal{A}} Q^\pi(s, a) \pi(da|s)$.

Dynamic programming exploits these recursive equations to improve policies iteratively. Starting from an initial policy π_0 , one alternates between two steps:

Policy evaluation. Given a policy π_k , compute its *Q-function* Q^{π_k} by solving the Bellman equation

$$Q^{\pi_k}(s, a) = r(s, a) + \gamma \int_{\mathcal{S}} \int_{\mathcal{A}} Q^{\pi_k}(s', a') \pi(da' | s') p(ds' | s, a).$$

This evaluation step corresponds to finding the fixed point of a contraction operator. It is performed iteratively, starting from a fixed function Q , typically $Q = 0$, and repeatedly applying the Bellman update for Q^{π_k} .

Policy improvement. Define a new policy π_{k+1} by choosing, at each state, an action maximizing the one-step lookahead based on Q^{π_k} :

$$\pi_{k+1}(s) \in \arg \max_{a \in \mathcal{A}} Q^{\pi_k}(s, a),$$

essentially replacing the current action in state s by one that has a higher value.

This procedure is called *policy iteration*.

Repeated policy evaluation and improvement steps eventually converge to an optimal policy. The corresponding optimal value function V^* then satisfies the *Bellman optimality equation*

$$V^*(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \int_{\mathcal{S}} V^*(s') p(ds'|s, a) \right\}. \quad (6)$$

This viewpoint will be particularly useful in the next section: many RL methods can be interpreted as approximate forms of policy iteration, where either the policy evaluation step, the policy improvement step, or both, are carried out only approximately from data [17].

Remark. The above procedure assumes that the expectations appearing in the Bellman equations can be computed exactly. This is rarely the case in general continuous systems, where these integrals are typically intractable. In finite state-action spaces, however, they reduce to finite sums, yielding an algorithm with polynomial complexity in the size of the state and action spaces. When the state-action space is very large, for example when it arises from a fine discretization of a d -dimensional continuous state space, the computational complexity of dynamic programming grows exponentially with d . This phenomenon, known as the curse of dimensionality [12], makes exact solutions of the Bellman equation intractable except for low-dimensional problems. Practical control methods therefore rely on additional structure to obtain scalable solutions. In this tutorial, we present two such approaches. In adaptive control, a reference model and a structured controller parametrization lead to tractable adaptation laws with stability guarantees. In RL, policies and/or value functions are represented within a restricted function class, enabling optimization directly in a lower-dimensional parameter space.

2.5 Divergence and convergence of the two fields

Dynamic programming is the common cornerstone of both RL and optimal control. However, it is important to note that while RL is almost synonymous with seeking optimality, the control community historically views optimality as secondary to stability. In fact, the literature is replete with examples of control designs that are nominally optimal but lack robust stability—culminating in the celebrated shortest abstract in history [23]—and are thus often unusable in engineering applications.

In this section, we highlight cultural and methodological divergences between the fields that may help explain how the solution methods in the next section were developed.

Divergent paths: in-silico vs. in-vivo. Despite the shared dynamic programming origins, the two fields have diverged based on their primary *environments*. On the one hand, RL has been developed largely within computer science and has traditionally focused on an agent interacting with an *in-silico* simulator. This sandbox environment supports an emphasis on algorithm design and permits millions of interactions with the simulator, often without stability or safety constraints. Consequently, algorithm convergence rates, sample efficiency, and the exploration-exploitation tradeoff (see Section 4.2) are primary metrics of success.

On the other hand, control methods emerged from engineering applications and are consequently designed for deployment in the “real world” and *in-vivo*. Here, the “agent” is a controller for a physical plant. In this context, optimality is often a luxury. The priorities are stability, safety (synonymous with constraint satisfaction), robustness to unmodeled disturbances or uncertainties, and implementation concerns involving real-time computation, partial observations, parsimonious use of data, and so on.

Cultural epistemology: certificates vs. benchmarks. This divergence and the different environments have fostered distinct research cultures. In the control community, a “valuable contribution” typically requires formal certificates—mathematical guarantees of safety or stability for the real-world closed-loop system.

These are usually derived through pen-and-paper analysis (e.g., Lyapunov stability), and computational certificates have been widely embraced only relatively recently. Conversely, the RL community prioritizes algorithmic certification, empirical performance, and scalability. Rather than tailoring solutions to domain-specific physics, RL has embraced the infamous *bitter lesson* [88]: the observation that general-purpose methods that leverage massive computation (e.g., deep RL) eventually outperform human-engineered features. Success in RL is thus typically defined by performance on standardized benchmarks or solving complex, high-dimensional tasks that defy classical approaches.

At the same time, modern control theory has deepened its understanding of robustness, safety, and performance guarantees for uncertain and nonlinear systems. Key developments, including robust adaptive control, Lyapunov-based analysis for learning systems, and learning-based model predictive control, have begun to address many of the limitations that once hindered adaptive systems in practice [5, 20, 40]. Importantly, these developments show that learning and safety need not be at odds, and that adaptive behavior can be both data-driven and certifiable when properly designed [1].

Recent confluence. These differences between RL and control are to be expected; they are logical consequences of their respective development in in-silico digital sandboxes and in-vivo physical systems. While their environments necessitated different priorities, the boundaries are now blurring. Modern control has increasingly incorporated learning-based components, while RL has begun addressing safety and robustness in real-world systems. These trends suggest that these two histories are once again merging into a unified field of data-driven decision-making.

3 Data-driven decision making

Both RL and adaptive control address the same fundamental challenge: how to design controllers when the system dynamics are not fully known. Their primary difference lies in how uncertainty is modeled and handled, as well as in the types of guarantees that are sought. A key bridge between the two fields is the literature on *approximate dynamic programming* [14, 58, 76], which combines function approximation with dynamic programming principles.

This core technical section first reviews the model-reference adaptive control algorithm as well as the actor-critic family of RL algorithms. We then present an original approach that combines both points of view in a low-level/high-level hybrid control strategy (Section 3.4). This new approach shows that the two paradigms can indeed be complementary rather than opposed.

3.1 Adaptive control

Adaptive control is the classical control-theoretic paradigm for updating a controller online in response to uncertain or changing dynamics while maintaining a stability-oriented design viewpoint. Among the many variants developed in this literature, MRAC [3, 34, 78] plays a particularly central role: one first specifies a stable reference model that encodes the desired closed-loop behavior, and then adjusts controller parameters in real time so that the plant tracks this reference behavior despite parametric uncertainty.

Figure 1 highlights the basic MRAC architecture. The reference trajectory r_k is filtered through a linear reference model with matrices A_m and B_m , defining the desired response $s_{m,k}$. The physical system dynamics are represented by a nominal linear input channel together with unknown state dynamics A , unknown input scaling Λ , and a nonlinear term approximated through basis functions $W^T \phi(\cdot)$. The matrix B (action directions) is assumed to be known. The actions are given by an adaptive controller that combines a feedforward gain K_r , a feedback gain K_s , and a nonlinear compensation term $W^T \phi(\cdot)$. The tracking error $e_k \doteq s_k - s_{m,k}$ between the realized system state and the reference-model state is then used in the adaptation law to update these parameters online, with the goal of keeping all closed-loop signals bounded and driving the tracking error to zero despite uncertainty in the system dynamics.

The key insight behind this architecture is the *matching assumption*, namely that there exist ideal feedback gains K_s^* and K_r^* satisfying

$$A - B\Lambda K_s^* = A_m, \quad B\Lambda K_r^* = B_m.$$

These conditions imply that the closed-loop system can, in principle, reproduce the target dynamics specified by the reference model. They are the standard matching conditions in adaptive control [68, 78]; in RL terminology, they play a role similar to a realizability assumption [66], namely that an optimal policy exists within the considered model class. Since the plant matrices are unknown, the ideal gains K_s^* and K_r^* cannot be computed directly. Instead, the adaptive controller maintains estimates K_s , K_r , and W , and it is convenient to express them relative to the ideal parameters. Under the matching assumption, the combined system, controller, and reference model yield the tracking-error dynamics

$$e_{k+1} = A_m e_k + B\Lambda \Theta^T \omega_k, \quad \text{with} \quad \Theta \doteq \begin{pmatrix} K_s^T - K_s^{*T} \\ K_r^T - K_r^{*T} \\ W - W^* \end{pmatrix}, \quad \text{and} \quad \omega_k \doteq \begin{pmatrix} s_k \\ r_k \\ \phi(s_k) \end{pmatrix}, \quad (7)$$

where Θ collects the parameter estimation errors. Denoting the dimensionality of s_k by d_s , of a_k by d_a , of r_k by d_r , and of $\phi(s_k)$ by d_ϕ , we have $e_k \in \mathbb{R}^{d_s}$, $\Theta \in \mathbb{R}^{(d_s+d_r+d_\phi) \times d_a}$, and $\omega_k \in \mathbb{R}^{d_s+d_r+d_\phi}$.

An adaptive controller now chooses parameter updates Θ_{k+1} such that the error dynamics (7) remain bounded and decay asymptotically. Given positive definite matrices $Q \in \mathbb{R}^{d_s \times d_s}$ and $\Gamma \in \mathbb{R}^{(d_s+d_r+d_\phi) \times (d_s+d_r+d_\phi)}$, the update law is typically derived from a Lyapunov function U such as

$$U(e, \Theta) = e^T P e + \text{tr}(\Theta^T \Gamma^{-1} \Theta \Lambda),$$

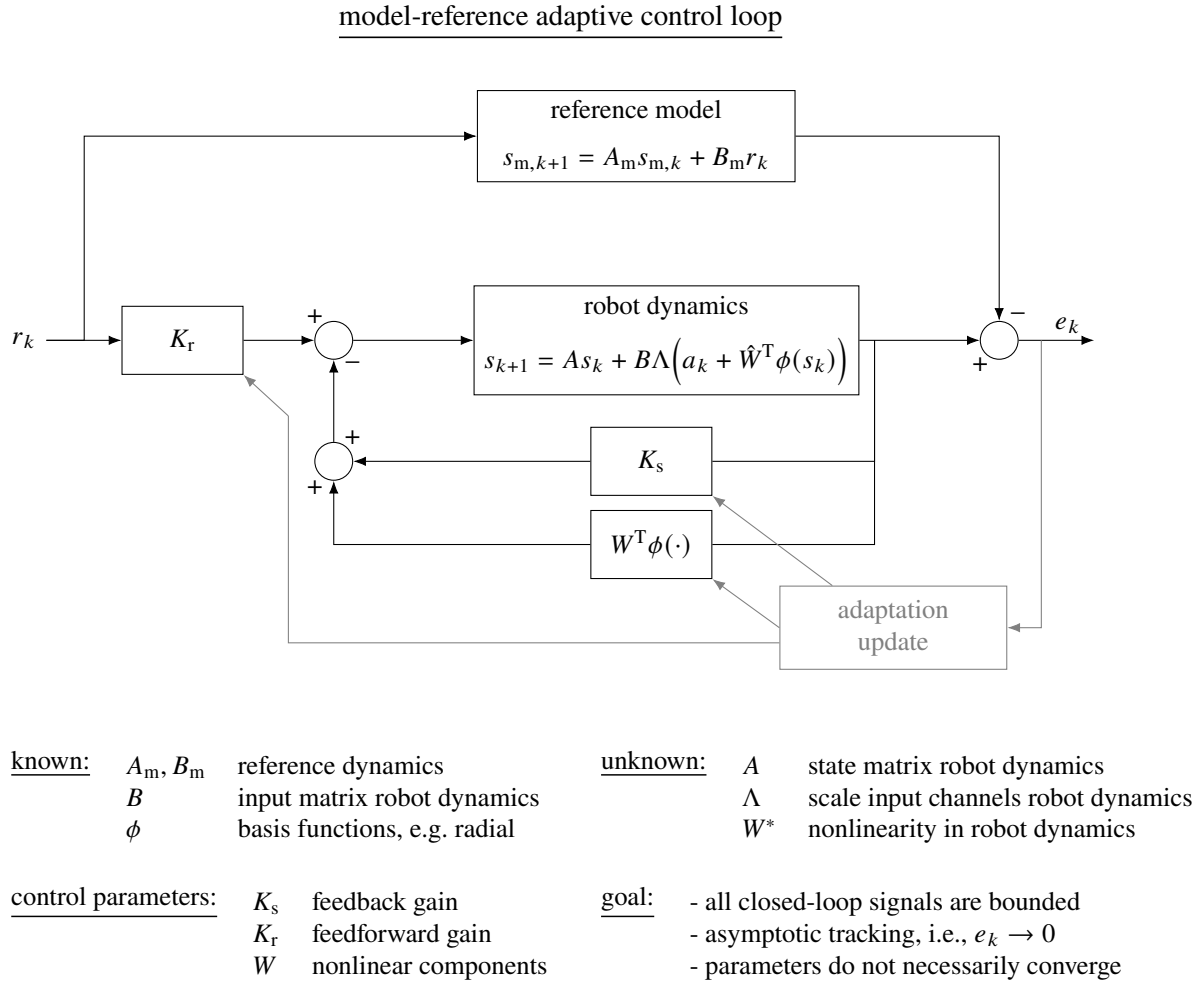


Figure 1: Architecture of a model-reference adaptive controller. The adaptation loop is shown in gray.

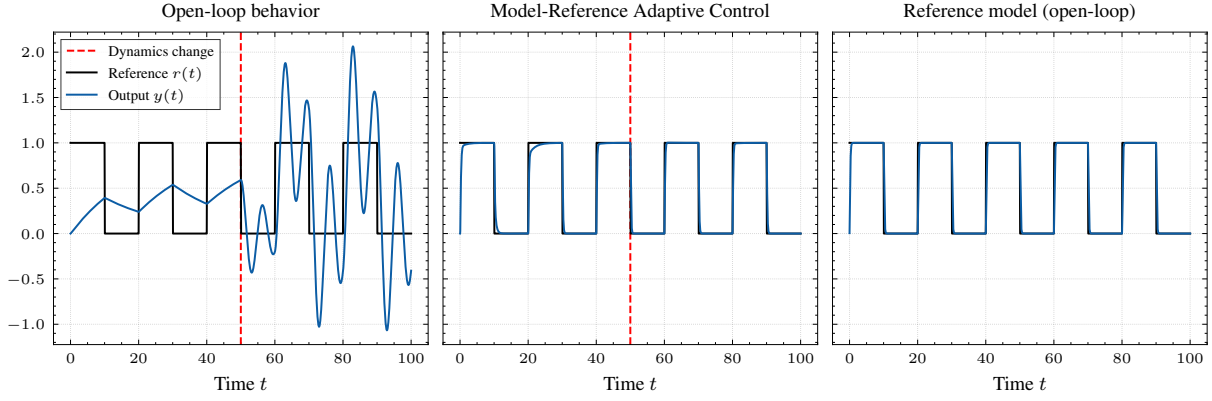


Figure 2: Illustration of model-reference adaptive control on a second-order system with a dynamics change. The goal is to make the output track the reference signal. (Left) Open-loop behavior of the system. The change in damping at $t = 50$ drastically changes the behavior. (Middle) MRAC automatically tunes the parameters of a feedback controller to accurately track the reference signal despite the dynamics change. (Right) Open-loop behavior of the reference model used by MRAC.

where P solves the Lyapunov equation $A_m^T P A_m - P = -Q$. The update law is chosen so that the Lyapunov function decreases along the trajectories of the adaptive system. For the case $d_a = \Lambda = 1$, this update becomes

$$\Theta_{k+1} = \Theta_k - \Gamma \omega_k B^T P e_k.$$

In continuous time, one can show that the corresponding decrease in the Lyapunov function is proportional to $-e^T Q e$, implying that the tracking error decreases over time.

In discrete time, the same calculation introduces additional higher-order terms in the sampling time and step size Γ .

Neglecting these terms, the update yields the approximate decrease

$$U(e_{k+1}, \Theta_{k+1}) - U(e_k, \Theta_k) \approx -e_k^T Q e_k,$$

suggesting that the tracking error contracts over time. Rigorous discrete-time analyses account for the higher-order terms explicitly and typically require sufficiently small adaptation gains, normalization, projection, or related modifications to guarantee boundedness and asymptotic tracking.

Even though the parameter update is expressed in terms of the parameter error Θ , the ideal parameters K_s^* , K_r^* , and W^* cancel when the update is translated to the parameter estimates and therefore do not need to be known.

The update law also admits a gradient-descent interpretation. If one considers an instantaneous quadratic tracking cost such as $e_k^T P e_k$, then the term $\omega_k B^T P e_k$ can be viewed as a sensitivity-weighted descent direction with respect to the controller parameters. Thus, at an intuitive level, MRAC updates the controller parameters to reduce the current tracking error in much the same way that online gradient descent would. In the derivation above, though, the update is selected not only for error reduction, but also because it supports a dissipation inequality.

Illustration: Second-order system. We now demonstrate the MRAC algorithm in a simple disturbance-free second-order dynamical system. Figure 2 illustrates such a system, in which the dynamics change at $t = 50$ from overdamped ($\zeta = 10$, “high friction”) to underdamped ($\zeta = 0.1$, “low friction”). The natural angular frequency of the system remains constant at $\omega = 1$, and the sampling period for the zero-order hold discretization is $T = 0.01$. Our goal is to make the output of this system (the displacement) follow the reference trajectory. In the Half-Cheetah joint example (1), this can be interpreted as $r(t)$ specifying the desired angle of the joint at time t , while $y(t) = \theta(t)$ is the actual angle. If the reference signal is directly fed as an input to the system, the tracking error is very large (this is shown in the left plot). To

accurately track the reference signal, feedback is necessary. Writing θ for the output, the general form of the input u applied to the system is

$$u(t) = k_r r(t) + k_\theta \theta(t) + k_{\dot{\theta}} \dot{\theta}(t).$$

Plugging this into (1), with $u = \tau/I$ and $\eta = 0$, the closed-loop joint dynamics are then given by

$$\ddot{\theta} + (2\zeta\omega - k_{\dot{\theta}})\dot{\theta} + (\omega^2 - k_\theta)\theta = k_r r.$$

Thus, by choosing the controller parameters k_r , k_θ , and $k_{\dot{\theta}}$ appropriately, the closed-loop system can follow any desired second-order dynamics. We now use the MRAC update to tune these parameters. For this, we first need to specify a reference model in which the open-loop behavior follows the reference trajectory closely. In our case, we choose a second-order system with natural frequency $\omega = 10$ (fast response) and $\zeta = 1$ (critical damping). The open-loop behavior of this system is illustrated on the right in Figure 2. The center plot shows the trajectory of the real system under the MRAC controller with $\Gamma = 100I$. Note that the controller parameters undergo a rapid change at $t = 50$ to enable tracking the reference trajectory despite the change in dynamics.

Limitations. Despite its elegance, MRAC has well-known limitations. These shortcomings motivated a substantial literature on robust and modified adaptive laws, including normalization, projection, dead zones, and composite adaptation, designed to improve robustness while preserving the core stability-oriented philosophy of adaptive control [42].

Take-home message. MRAC captures the canonical gray-box perspective of adaptive control emphasized throughout this paper.

The reference model specifies the desired evolution of each joint angle along a target gait trajectory, while the adaptive law adjusts controller gains online so that the realized angles continue to track these references even when the robot’s effective dynamics are altered by changes in link masses, joint friction, or other modeling mismatches. This decomposition is attractive because it preserves a familiar robotics control architecture—trajectory generation at a higher level and fast joint-level feedback underneath—while using adaptation to compensate for structured uncertainty at the actuator and joint-dynamics level.

3.2 Adaptive control beyond MRAC

Beyond classical MRAC, adaptive control has developed along three major directions: (i) extension to increasingly complex system classes, including nonlinear, distributed-parameter, and delay systems; (ii) improved transient performance and robustness to modeling errors and disturbances; and (iii) understanding the role of excitation for learning and control.

A major step beyond linear MRAC was the introduction of adaptive backstepping [52], a recursive Lyapunov-based methodology for nonlinear systems. Unlike classical MRAC, which primarily provides asymptotic guarantees, adaptive backstepping enabled systematic controller design for broad classes of nonlinear plants and led to techniques for improving transient performance under large parametric uncertainty. Subsequent developments extended these ideas to systems with unmodeled dynamics and external disturbances.

The same philosophy was later generalized to increasingly complex dynamical systems. In particular, backstepping-based methods were adapted to control partial differential equations (PDEs), yielding explicit controller constructions for distributed-parameter systems. When unknown plant parameters are estimated online and incorporated into these controllers, one obtains adaptive control schemes for PDEs [83]. Related developments led to adaptive control methodologies for systems with unknown delays, where online estimation of delays can be incorporated into the controller design [105].

Another active line of research concerns robustness to non-parametric uncertainties such as disturbances and unmodeled dynamics. Classical mechanisms such as leakage and projection guarantee boundedness of the adaptive parameters but may introduce residual regulation errors that depend on unknown quantities.

More recent approaches seek robustness guarantees that are independent of the magnitude of the parametric uncertainty. One recent example is deadzone-adapted disturbance suppression [46], which provides arbitrarily small regulation errors despite large parametric uncertainty.

Finally, adaptive control has long emphasized a key issue that is closely related to exploration in reinforcement learning (see Sec. 4.2): the need for informative data. Accurate parameter estimation typically requires *persistent excitation*, a condition ensuring that the observed trajectories contain sufficient information to identify unknown parameters. However, generating such excitation may conflict with the primary control objective of regulating the system. This tension is analogous to the exploration-exploitation tradeoff in reinforcement learning, where informative actions may temporarily degrade performance but improve future decision making. A notable achievement of adaptive control is that many adaptive controllers achieve strong regulation and tracking guarantees without requiring persistent excitation [8, 68]. Rather than aiming to identify the system completely, these methods learn only the information necessary to achieve the control objective. This perspective is also present in the RL literature, where exploration through optimism (see Sec. 4.2) aims to collect only the amount of data needed to solve the reward-optimization problem.

3.3 A standard RL approach: actor-critic algorithms

In this section we outline a canonical class of RL algorithms, called actor-critic. We will first present a more basic version, then discuss some of the specific implementation choices that matter when using neural networks (i.e., the deep RL setting). We consider a setting where no white-box model nor high-fidelity simulator of the environment is available. Rather, we only have data, and potentially some knowledge of the problem.

The goal is to learn a parameterized policy $\pi_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$ that minimizes the objective introduced in Equation (2), or equivalently maximizes the expected discounted reward from the start states: $V(\pi) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$.

To improve the policy, the agent must estimate which actions lead to high long-term rewards using the action-value function $Q^\pi(s, a)$ (Equation (5)). Modern RL algorithms typically learn an approximation of this quantity and use it to guide policy improvement.

For example, π_θ could be a neural network with parameters θ that takes the state s as input and outputs a multidimensional Gaussian distribution over actions.

A key result underpinning actor-critic methods is the policy gradient theorem [89]. It shows that the gradient of the objective can be expressed without differentiating through the environment dynamics, relying only on the policy and its action-value function:

$$\nabla_\theta V(\pi) = \int d_\gamma^\pi(s) \int \pi_\theta(a|s) Q^{\pi_\theta}(s, a) \nabla \ln \pi_\theta(a|s) da ds.$$

The term $d_\gamma^\pi(s)$ corresponds to the discounted state-visitation distribution under policy π .³

The policy gradient theorem suggests a simple strategy: sample states and actions from the current policy and weight the policy update by the corresponding action-value.

Obtaining unbiased samples of this gradient, as in algorithms such as REINFORCE [100], is typically not sample efficient; on-policy algorithms such as proximal policy optimization obtain nearly unbiased samples [79]. Instead, many algorithms estimate the action-value function to obtain a biased approximation of the gradient from a state s .

³Note that the definition of $d_\gamma^\pi(s)$ is slightly different between the episodic and continuing settings in RL. In the continuing setting, $d_\gamma^\pi(s)$ corresponds to the stationary distribution for the policy π , with no dependence on γ . Otherwise, the rest of the gradient remains the same. Due to this similarity, algorithms designed for the episodic setting can often be applied heuristically to the continuing setting, although they should account for differences in state weighting. To clarify terminology, the *episodic* setting corresponds to *batch control* problems and the *continuing* setting corresponds to the *continuous* setting in control. This terminology should not be confused with the fact that in RL the continuous setting usually means the state and/or action are continuous rather than discrete.

Actor-critic methods replace the unknown quantity $Q^{\pi_\theta}(s, a)$ appearing in the policy gradient theorem with a learned approximation $q_w(s, a)$. The policy π_θ is referred to as the *actor*, while the learned action-value approximation q_w is called the *critic*. The actor chooses actions and the critic evaluates their long-term consequences. Conceptually, this mechanism can be viewed as a form of approximate policy iteration. The critic approximates the value for the current policy (actor) and the actor uses these approximations to improve the policy.

Specifically, the algorithms use the gradient approximation at state s

$$\Delta\theta \doteq q_w(s, a) \nabla \ln \pi_\theta(a|s) \quad \text{using } a \sim \pi(\cdot|s). \quad (8)$$

The critic q_w gives an estimate of the expected return (value) from the current policy, instead of having to run the current policy for multiple steps (a rollout) to get an unbiased sample of the return. As the policy π_{θ_t} changes with parameters θ_t on time step t , q_{w_t} is also updated to track an estimate of the action-values for π_{θ_t} .

Illustration on the Half-Cheetah. Let us consider more precisely how this algorithm behaves while interacting with our running example, Half-Cheetah. The agent starts in an initial state s_0 and samples an action a_0 from its stochastic policy π_{θ_0} , which is typically set to be a Gaussian policy. The action space is bounded, so a tanh parameterization is used to respect the action bounds: $\pi_\theta(a|s) = \tanh(g_\theta(s, \epsilon))$, where $g_\theta(s, \epsilon) = \mu_\theta(s) + \epsilon \sigma_\theta(s)$ and $\epsilon \sim \mathcal{N}(0, I)$ is a vector of normal variables, one for each action dimension; $\mu_\theta(s)$ and $\sigma_\theta(s)$ are the outputs of the neural network. If there are multiple actions, it is typical to use independent Gaussians for each action. Note that, despite this choice, the joint utility of actions is still considered, because the critic evaluates the joint utility of all actions. The primary limitation of using independent Gaussians is that they do not allow for correlations when sampling actions, though the mean can still converge to the optimal multivariate action in each state.

After taking action a_0 , the environment transitions to state s_1 and the agent observes reward r_1 . The agent adds this transition (s_0, a_0, r_1, s_1) to its replay buffer. It is typical to wait for the replay buffer to fill for some number of steps before updating, meaning that the agent takes random actions during the first few steps. If previously collected offline data are available, the agent can initialize its replay buffer with these historical data and omit this initial exploration phase. In our Half-Cheetah experiment, for example, this exploration phase lasts 100 steps, and the maximum size of the replay buffer $\mathcal{D}$ is 100,000 samples.

Once the buffer has sufficient samples, both the actor and critic are updated at each step using sampled minibatches $\mathcal{B} \subset \mathcal{D}$ from the replay buffer $\mathcal{D}$. For Half-Cheetah, a typical minibatch size is $b = 256$, meaning that $\mathcal{B}$ contains 256 tuples (s, a, r, s') randomly selected from $\mathcal{D}$. The action-value function (critic) q_w is updated using a TD update on these tuples:

$$\Delta w \doteq \frac{1}{b} \sum_{(s, a, r, s') \in \mathcal{B}} (r + \gamma q_w(s', a') - q_w(s, a)) \nabla q_w(s, a). \quad (9)$$

The policy is updated using the same minibatch, though only the state is used:

$$\Delta\theta \doteq \frac{1}{b} \sum_{s \in \mathcal{B}, a \sim \pi_\theta(\cdot|s)} (q_w(s, a) - v(s)) \nabla \ln \pi_\theta(a|s). \quad (10)$$

For this *log-likelihood* (score function) form of the gradient, it is typical to include an estimate $v(s)$ of the value function $\mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[q_w(s, a)]$. This centered term, $q_w(s, a) - v(s)$, reflects the *advantage* of one action over another in the given state. It is often used as a heuristic to provide a more effective gradient update because it is negative for low-valued actions and positive for high-valued actions. When the policy distribution can be reparameterized—as it can be for the Gaussian—and the critic q_w is differentiable with respect to the input actions, a typically more effective alternative is the *reparameterized* gradient update obtained by differentiating $q_w(s, \tanh(g_\theta(s, \epsilon)))$ with respect to θ [36]. These minibatch stochastic gradient updates for the critic and actor are applied at each step—potentially multiple times per step—using standard machine-learning optimizers such as Adam [49], which uses vector step sizes and

momentum. The replay buffer is a sliding window, with the oldest samples dropped as new interactions are added.

The above update strategy forms the basis of many actor-critic updates in deep RL, albeit with a variety of implementation heuristics introduced to improve performance. Soft actor-critic (SAC) [36] uses entropy regularization to maintain stochasticity in the policy, learns two critics to reduce overestimation in the critic’s bootstrap target, uses a slowly changing target to promote stability, and applies the reparameterized gradient update to the actor. The approach can also be extended to learning deterministic policies with an algorithm called DDPG [80] and a corresponding variant, TD3 [30], which similarly adds a variety of implementation heuristics to improve performance.

Take-home message. Actor-critic methods exploit differentiable parametrized representations of the policy and action-value functions. At each update, an approximate gradient of the optimization objective is computed directly using data from previous rollouts stored in the replay buffer. Concretely, these data are used to update the representations of the actor π_θ and the critic q_w . Because these algorithms do not need to estimate a model of the dynamics, they are called *model-free*. They are designed to learn optimal behavior from scratch in terms of the cumulative-reward objective, without using reference trajectories or demonstrations.

Limitations. These algorithms lack some critical components of the RL problem setting: directed exploration, handling partial observability, and, in many cases, even soundness. We discuss exploration and partial observability in the next section but focus on soundness here. The primary issue is that these algorithms are largely based on heuristics. Their updates use biased gradients,

and in some cases these are known to diverge [96] or converge to suboptimal solutions [35, 73, 94]. Typically, the state weighting $d^\pi_\gamma(s)$ is ignored, and updates are instead drawn randomly from the replay buffer, which has a different implicit distribution. This mismatch can cause suboptimal solutions, though there is some theoretical and empirical evidence that, with sufficiently high-capacity (large) neural networks, it does not cause issues [35]. Another key issue arises when moving from linear function approximation, for which we have more theoretical guarantees [18, 47, 103], to nonlinear function approximation with neural networks in deep RL. For the critic, the TD update is known to diverge under nonlinear function approximation [96]. A variety of gradient TD approaches have been proposed to remedy this problem [73], along with some theoretical results for actor-critic methods with neural networks [32]. These more principled approaches, however, have not yet been widely adopted, and commonly used standard algorithms such as SAC and TD3 have no such guarantees. These issues are recognized, and progress is being made toward sounder and more robust variants of actor-critic methods that use neural networks. Today, however, these algorithms must still be used with caution and have not reached the same level of maturity as RL algorithms for tabular or linear settings or as algorithms in control.

Another limitation of these algorithms is that they are designed to find the optimal policy without considering performance during learning. Following the direction of the gradient to optimize the objective does not ensure good transient performance. Relatedly, stability and constraints are not encoded in this objective, and any stability or safety properties must be encoded in the costs themselves; even then, they could be violated during learning. At the same time, it is worth noting that finding the optimal policy is not strictly at odds with good transient behavior. Acting nearly greedily does push the algorithm toward reducing cost sooner. The agent adjusts based on feedback from its interaction with the environment and will reduce costs, at least locally. The RL literature includes many works that report transient behavior and rank algorithms accordingly (see [74]), despite the fact that the objective itself does not optimize for transient behavior. Finally, there is a growing literature on safe RL that provides guarantees on constraint satisfaction during learning [41, 85, 99] or uses offline data to learn control barrier functions [93].

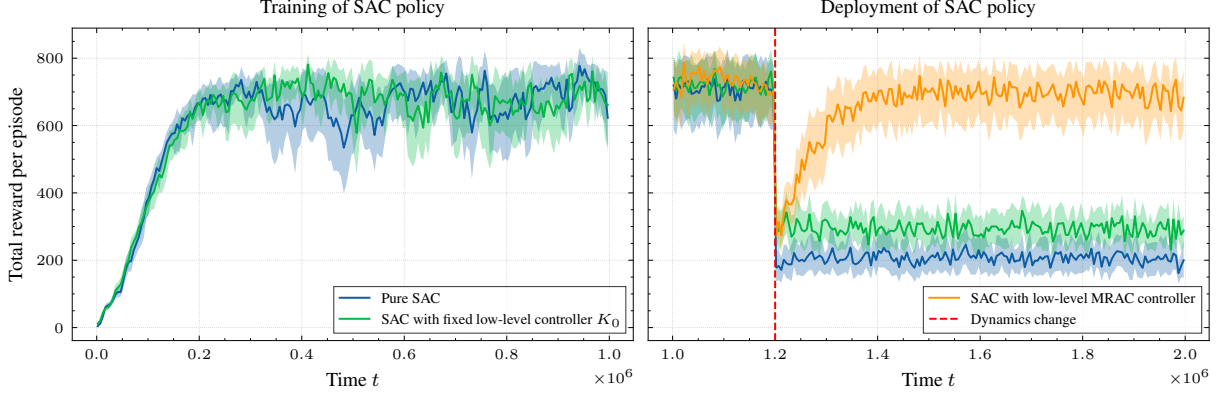


Figure 3: SAC in the Half-Cheetah environment. The left plot shows the first 10^6 time steps, in which an SAC policy is trained. The right plot shows the second 10^6 time steps, in which the learned SAC policy is deployed. After 200,000 steps of deployment, the friction in the Half-Cheetah joints changes, causing the non-adaptive policies to struggle. The orange line shows our proposed SAC+MRAC solution, in which a low-level adaptive controller is combined with a fixed SAC policy. The low-level MRAC controller aims to restore the nominal low-level dynamics encountered in the first 200,000 steps of deployment of the SAC- K_0 policy, on the basis of which a linear reference model is learned.

3.4 Combining RL and adaptive control

As described above, modern RL algorithms like SAC usually focus on learning a fixed policy that achieves high performance in a static environment. In this section, we examine how SAC can be combined with a low-level MRAC controller to adapt this fixed policy to changes in the environment dynamics. Training a policy in a static environment is straightforward, and Figure 3 (left, blue line) shows that SAC manages to make the Half-Cheetah run after about 200,000 samples from the environment, after which policy performance plateaus. If we deploy this fixed policy in an environment whose dynamics change, the performance will drop. This is illustrated on the right in Figure 3 (blue line), where, at $t = 1.2 \times 10^6$, the damping coefficients of all six Half-Cheetah joints are reduced by a factor of 2, as might occur after lubrication. The performance of SAC drops drastically because the applied torques do not account for the reduced friction of the joints. What has not changed, however, is the type of gait that the Half-Cheetah should follow. In other words, if the policy were to choose a set of desired joint angles instead of joint torques, then these angles would still be applicable after the dynamics change.

Following desired angles is a reference tracking problem, as in Section 3.1.

In this case, the reference signal $r(t)$ is the output of the SAC policy, and we apply a low-level controller with parameters k_r , k_θ , and $k_{\dot{\theta}}$ to compute the action $a(t)$ at time t :

$$a(t) = k_r r(t) + k_\theta \theta(t) + k_{\dot{\theta}} \dot{\theta}(t).$$

Modeling the joints again as second-order systems, the closed-loop dynamics are

$$\ddot{\theta} + (2\zeta\omega - k_{\dot{\theta}})\dot{\theta} + (\omega^2 - k_\theta)\theta = k_r r. \quad (11)$$

A reasonable start would be to set $k_{\dot{\theta}} = 0$ and $k_r = -k_\theta \gg \omega^2$. Then, the angles approximately follow

$$\ddot{\theta} \approx -2\zeta\omega\dot{\theta} - k_r(\theta - r),$$

meaning that the angle converges to r at a speed depending on k_r , ζ , and ω . Since the Half-Cheetah system requires actions a to lie in $[-1, 1]$, we cannot choose k_r arbitrarily high.

In Figure 3, we show how SAC behaves when combined with a fixed low-level controller K_0 , in which $k_{\dot{\theta}} = 0$ and $k_r = -k_\theta = 1$. During training and deployment in the original environment, the performance is on par with pure SAC. After the dynamics change, the performance is slightly better than pure SAC, but

not by much. Just like in the second-order system discussed in Section 3.1, a change in dynamics requires a change in control parameters. And just like in Section 3.1, we will use the MRAC algorithm to tune the parameters automatically.

Since the SAC- K_0 combination works well under the original dynamics, we can use this system to create a reference model. Specifically, we collect a dataset of 200,000 steps of interaction, recording $(r_k, \theta_k, \dot{\theta}_k, \theta_{k+1}, \dot{\theta}_{k+1})$ at every discrete time k , for each of the six joints. We then find a least-squares fit for the matrices A and B such that

$$\begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix}_{k+1} \approx A \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix}_k + B r_k$$

These matrices specify our reference model, and B is also used as the ‘real’ input matrix in the MRAC update. Note that if the original system dynamics obey Equation (11) and the change in dynamics affects only the damping coefficient ζ , it is only necessary to adjust the coefficient $k_{\dot{\theta}}$ to restore the original dynamics. We thus set the components of Γ corresponding to k_r and k_{θ} to zero and the component corresponding to $k_{\dot{\theta}}$ to 10^{-7} . Such a small step size is necessary to prevent instabilities that arise due to the neglected nonlinear nature of the system. Similarly, we clamp $k_{\dot{\theta}}$ to the interval $[-0.3, 0.3]$. Finally, the MRAC update depends on the error term $e = s - s_m$, where the model state s_m follows the reference dynamics. In our case, the reference dynamics are linear and given by the A and B matrices described above. As the real Half-Cheetah system exhibits highly nonlinear contact dynamics that are impossible to realize in the reference model, the error term becomes very large and uninformative. We thus replace the error e by the one-step prediction error, effectively resetting s_m to s at every time step. This is a significant change to the MRAC update derived in Section 3.1, which does not preserve theoretical guarantees that apply in the linear setting. We do not include a nonlinear $W^T \phi$ term in the controller. The final behavior is shown in Figure 3 (right, orange line). It can be seen that the adaptive controller restores the low-level dynamics to the nominal dynamics, which enables the SAC policy to effectively control the robot. Our SAC implementation uses the hyperparameters mentioned in Section 3.3. The code is published at <https://github.com/onnoeberhard/forlac-tutorial>.

Remark: An alternative approach would be to continue updating the SAC policy after the dynamics change. Adapting RL algorithms to nonstationary environments is an active area of research, involving questions such as how to detect changes in the environment, how to incorporate new experience while retaining useful past knowledge, and how to adapt sufficiently quickly to maintain performance. The goal of our example is not to suggest that a hybrid RL–adaptive control architecture is the only viable solution to such problems. Rather, it provides a simple and illustrative example of how the two methodologies can be combined. In this setting, the RL component learns a complex locomotion strategy, while the adaptive controller compensates online for changes in the system dynamics. The resulting architecture highlights the complementary strengths of the two fields and serves as a concrete illustration of the broader theme of this tutorial.

4 Additional considerations

In this section, we review and contrast a few additional key aspects of RL and control algorithms: the use of models, exploration, and the handling of partial observability.

4.1 Model-based versus model-free algorithms

Control has historically been mostly applied to well-known physical systems, and the predominant paradigm is that of a known gray-box model with parametric uncertainty. Access to a dynamics model, even only a gray-box model, is central in control theory and drives both algorithm design and the derivation of guarantees through Lyapunov analysis. Where such a first-principles model is a reasonable reflection of the deployment environment, this model can be used for powerful algorithms (e.g., model predictive control). It also facilitates incorporating constraints, and it can be instrumental in the analysis while providing some interpretability of the controller.

When no gray-box model of the environment is available, both fields have developed parallel approaches. While it is most common in RL to assume minimal prior knowledge of the system, we still make a distinction between *model-based* and *model-free* algorithms. In *model-based RL*, a model of the system dynamics is learned from data and subsequently used for planning, typically via dynamic programming or decision-time optimization [87]. The model can be used for multi-step rollouts (planning) or for improving value and policy estimates. In contrast, *model-free RL* algorithms aim to directly estimate value functions, policies, or their gradients from data, without explicitly constructing a model of the environment, similarly to actor-critic methods presented in the previous section. In control, similar paradigms are known as *direct* or *indirect* control. The process of learning a model offline through data collection is known as *system identification*, which can be compared to the model-based RL approach. An important difference, though, is that model-based RL algorithms usually do not attempt to estimate a full model but rather only what is useful for the desired objective, very much in line with the recent developments in adaptive control discussed in Sec. 3.2.

The distinction between model-based and model-free methods in RL, however, is not always sharp. In particular, the replay buffer used in many model-free methods can be interpreted as a form of nonparametric model of the dynamics [71, 98]. From this perspective, updating value functions using stored transitions resembles planning with an implicit model defined by the data. Moreover, in complex environments where accurate parametric models are difficult to learn, replay buffers can be competitive with, or even outperform, learned parametric models [2]. There have been continued efforts to leverage learned models more effectively in RL, including methods that combine model learning with value estimation and policy optimization [37, 38, 57]. Nonetheless, the dominant paradigm remains to rely directly on data through replay buffers for repeated updates. Conversely, modern control actively reflects on using data together with model learning [22].

It is worth noting that, although RL is conceptually formulated as learning through interaction with an unknown environment, in practice it is often applied in settings where a simulator is available [11, 19, 21]. In such cases, the simulator effectively acts as a generative model that can be queried to produce data. However, the simulator is typically treated as part of the environment rather than explicitly exploited for planning, and many widely used algorithms remain model-free even in this setting.

4.2 Exploration and finite-time performance

Most RL algorithms are based on data collection using a current policy. This poses the problem of *coverage*: how can we guarantee that the data is sufficient to improve the policy? This issue already appears in classical convergence results for tabular RL algorithms such as Q-learning and SARSA [97]. These guarantees typically rely on a persistent exploration condition: all state-action pairs must be visited infinitely often. Simple strategies such as ϵ -greedy exploration (a form of uniform random persistent excitation) enforce this condition and ensure asymptotic convergence under suitable assumptions [97]. However, they do not control how efficiently information is gathered and may require a large number of samples to achieve good performance. In this section, we discuss how regret minimization formalizes the problem of minimizing data usage in RL and describe more efficient exploration algorithms.

Finite-time performance and regret. From a statistical perspective, asymptotic convergence is not sufficient: one is interested in the performance of the algorithm after a finite number of interactions with the system, so we need to account for such data collection steps.

The notion of *regret* is central in RL theory and captures the performance of the learner during the transient regime compared to that of the optimal policy:

$$\text{Regret}(T) = \sum_{t=1}^T (V(\pi^*) - V(\pi_t)). \quad (12)$$

Minimizing regret requires balancing exploration and exploitation: exploring uncertain actions to improve future decisions, while exploiting current knowledge to achieve low immediate cost. This exploration-

exploitation trade-off has been a central topic of research at the intersection of RL, online learning, and statistical learning theory over the past two decades.

Optimism and efficient exploration. A key principle underlying many theoretically efficient RL algorithms is *optimism in the face of uncertainty* [9] (or OFU). The idea is to construct estimates of the value function together with confidence intervals on these estimates. Then, an optimistic proxy of the value function is constructed by combining raw estimates and uncertainty in a way that favors poorly explored state-action pairs, thereby encouraging the agent to gather information where it is most needed. This mechanism, which was discovered independently in the control and statistics communities [53, 54], leads to algorithms with provable regret guarantees [43].

Exploration in deep RL. In large or continuous state spaces, the tabular exploration strategies described above cannot be applied directly. Modern deep RL algorithms instead rely on a variety of heuristic mechanisms to encourage exploration while learning value functions or policies represented by neural networks. In discrete domains, a prominent example is the use of Monte-Carlo tree search with optimism-based exploration bonuses, as in the upper confidence tree algorithm [50]. This approach was a key ingredient in the AlphaGo and AlphaZero systems [81, 82], where neural networks approximate value and policy functions while tree search provides targeted exploration of promising actions.

Other approaches attempt to estimate uncertainty directly in the learned value function, for example through bootstrapped ensembles [70], posterior sampling [67, 69, 84], or intrinsic motivation signals based on prediction error or state novelty [10, 63]. While a large body of work has been devoted to extending these ideas beyond the tabular and linear settings [55], there is little, if any, work aimed at bridging the gap between theoretically sound algorithmic principles and heuristics used in practice.

Exploration and excitation in continuous control. In continuous control settings, exploration is often implemented by randomizing the policy during data collection, for instance by injecting noise into the control actions or by explicitly encouraging high-entropy policies. From a control perspective, this idea is closely related to the notion of *persistent excitation*, which ensures that the system is sufficiently stimulated to identify unknown parameters or dynamics. Recent work in RL has begun to revisit this connection [25], emphasizing that informative excitation of the system can play a crucial role in enabling efficient learning of dynamics and value functions in continuous environments [67]. However, designing exploration strategies that simultaneously guarantee stability, safety, and efficient learning remains an open problem at the intersection of RL and adaptive control.

4.3 Partial observability

Many techniques from both RL and control theory rely heavily on knowledge of the system *state*. Environments where the state is revealed at all times are called “fully observable.” This knowledge enables many techniques that would otherwise not be possible. The reason is that the state is *Markovian*: for predicting the future, there is no relevant information in the history that is not contained in the current state. Most realistic environments, however, are *partially observable*, meaning that only observations derived from the state are revealed. For example, in the Half-Cheetah system, the observations at time t might include only the angular velocities of the joints, not the corresponding angles. Thus, past observations have to be used to infer the full state; in this case, the angles can be obtained by integrating the angular velocities.

There are two fundamental ways to approach partially observable environments: estimating *belief states* or directly summarizing histories. In control theory, it is more typical to estimate the belief state from the stream of observations. If the observations y are randomly sampled from an emission distribution $p(y | x)$ conditioned on the state x , then this can be done by Bayesian filtering. In particular, the goal is to compute the belief distribution over the state x_t given the history of observations $y_0, \dots, y_t$. This can

be done recursively, as

$$p(x_t | y_0, \dots, y_t) = \frac{p(x_t | y_0, \dots, y_{t-1})p(y_t | x_t)}{\int p(x'_t | y_0, \dots, y_{t-1})p(y_t | x'_t) dx'_t},$$

and

$$p(x_t | y_0, \dots, y_{t-1}) = \int p(x'_{t-1} | y_0, \dots, y_{t-1})p(x_t | x'_{t-1}) dx'_{t-1}.$$

In practice, these integrals are often intractable to compute, but in the special case of a Gauss-Markov model, where the transition and emission probabilities are linear-Gaussian, these integrals can be computed analytically. In this case, this recursive computation results in the celebrated *Kalman filter*. Furthermore, in a linear-quadratic-Gaussian dynamical system, it is provably optimal to choose actions based only on the mean of the belief distribution. This strategy is called *certainty equivalence control*; in nonlinear systems, such a strategy is generally suboptimal. In many systems, the degree of uncertainty in the belief distribution matters, since it may be advantageous to execute “information-gathering” actions to reduce this uncertainty and enable a more informed decision about the optimal strategy.

Explicitly tracking the belief state is not very common in RL. There is a large literature in RL on partially observable Markov decision processes (POMDPs), which involves estimating belief states, but much of this work assumes discrete states, even when applied to continuous settings like those in robotics [56]. In general, the belief distribution is not only difficult to compute, but also difficult to handle. Further, it is not strictly necessary for good control: the history contains the same information as the belief distribution.

Thus, in RL, it is more common to summarize the history directly, either by using fixed memory mechanisms such as frame stacking [65] and exponential moving averages [26, 44], or by learning a recurrent neural network memory end-to-end, jointly with the policy [28, 45]. These approaches often work well in practice, despite the fact that the general partially observable RL setting is provably intractable, both computationally and statistically [51, 59, 72]. To explain this discrepancy, several recent works have identified classes of partially observable environments in which these worst-case results do not apply [27, 33, 60, 104]. Extending these classes is an important open problem in RL theory.

5 Conclusions and open problems

RL and optimal control share a common foundation in dynamic programming but have developed distinct perspectives shaped by their respective domains. RL emphasizes data-driven optimization and scalability, while control focuses on stability, robustness, and formal guarantees.

In this tutorial, we presented a unified view of the two fields by comparing their problem formulations, learning paradigms, and methodological tools. We discussed how adaptation arises through RL and adaptive control, and clarified the role of models and data in model-based and model-free approaches.

Our main takeaway is that many differences between the two fields stem from differing assumptions and priorities rather than fundamentally different problems. As both communities increasingly address complex, real-world systems, their methods are converging toward a shared framework for data-driven decision making that balances performance with reliability.

Open Problems. In addition to the specific open problems discussed in the sections above, we mention a few research areas where we believe that synergies between the RL and control communities could be particularly valuable. One such area is long-horizon planning in robotics, where agents must reason over extended time scales while still satisfying short-term stability and safety requirements. Another is the design of hierarchical architectures that integrate high-level decision making with low-level execution, allowing abstract plans to be translated into reliable motor commands or process-level control actions. A closely related challenge is to understand how learning should be distributed across these levels: low-level controllers may adapt to uncertain dynamics and disturbances, while high-level policies may learn task structure, constraints, and strategies for exploration. This also raises the question of how experience can be consolidated across time scales, for instance through model-based rollouts, replay,

or “dreaming” mechanisms that allow an agent to refine its behavior without direct interaction with the physical system. Further open problems include partial observability, where controllers must act based on incomplete or noisy measurements, and robust reinforcement learning, where policies must retain reliable performance under distribution shifts, modeling errors, and adversarial or unexpected disturbances. Importantly, for real-world deployment, safety (i.e., hard constraint satisfaction) is non-negotiable, which poses a formidable challenge for any policy, whether derived using RL or classical control. More broadly, many real-world systems require a careful interplay between feedforward and feedback mechanisms: feedforward components can anticipate desired behavior or compensate for predictable effects, while feedback controllers remain essential for robustness to uncertainty, disturbances, and modeling errors. Progress on these questions will further consolidate the impact of RL and control theory on complex technological, social, and physical systems, ranging from robotics and industrial processes to foundation models and agentic artificial intelligence.

Acknowledgments

C. Vernade thanks the German Research Foundation (DFG) for its support through project 468806714 of the Emmy Noether Programme and under Germany’s Excellence Strategy—EXC number 2064/1, project number 390727645. C. Vernade also gratefully acknowledges funding from the European Union (ERC, ConSequentIAL, 101165883). Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them. Claire Vernade and Onno Eberhard thank the International Max Planck Research School for Intelligent Systems for its support. Martha White would like to thank the Natural Sciences and Engineering Research Council of Canada (NSERC), the Canada Research Chair program, the Canada CIFAR AI Chair Program, and the Alberta Machine Intelligence Institute for research support. Csaba Szepesvári gratefully acknowledges funding from the Canada CIFAR AI Chairs Program, Amii, and NSERC. Michael Muehlebach thanks the German Research Foundation for its support.

References

- [1] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017.
- [2] F. Aminmansour, T. Jafferjee, E. Imani, E. J. Talvitie, M. Bowling, and M. White. Mitigating value hallucination in Dyna-style planning via multistep predecessor models. *Journal of Artificial Intelligence Research*, 80:441–473, 2024.
- [3] B. D. Anderson, R. R. Bitmead, C. R. Johnson Jr, P. V. Kokotovic, R. L. Kosut, I. M. Mareels, L. Praly, and B. D. Riedle. *Stability of adaptive systems: Passivity and averaging analysis*. MIT Press, 1986.
- [4] B. D. O. Anderson and J. B. Moore. *Linear Optimal Control*. Prentice Hall, 1971.
- [5] A. M. Annaswamy. Adaptive Control and Intersections with Reinforcement Learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 6:65–93, 2023.
- [6] A. M. Annaswamy and A. L. Fradkov. A historical perspective of adaptive control and learning. *Annual Reviews in Control*, 52:18–41, 2021.
- [7] K. Åström. History of adaptive control. *Encyclopedia of Systems and Control*, pages 902–909, 2021.
- [8] K. J. Åström and B. Wittenmark. *Adaptive Control*. Dover Publications, 2nd edition, 2013.
- [9] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time Analysis of the Multi-armed Bandit Problem. *Machine Learning*, 47:235–256, 2002.
- [10] M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos. Unifying count-based exploration and intrinsic motivation. *Advances in Neural Information Processing Systems*, 29, 2016.
- [11] M. G. Bellemare, S. Candido, P. S. Castro, J. Gong, M. C. Machado, S. Moitra, S. S. Ponda, and Z. Wang. Autonomous navigation of stratospheric balloons using reinforcement learning. *Nature*, 588(7836):77–82, 2020.

- [12] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [13] D. P. Bertsekas. *Dynamic Programming and Optimal Control*. Athena Scientific, 4th edition, 2017.
- [14] D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.
- [15] S. Boyd, L. El Ghaoui, E. Feron, and V. Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, 1994.
- [16] W. I. Caldwell. Control system with automatic response adjustment, 1950. US Patent 2,517,081.
- [17] A. Chan, H. Silva, S. Lim, T. Kozuno, A. R. Mahmood, and M. White. Greedification operators for policy optimization: Investigating forward and reverse KL divergences. *Journal of Machine Learning Research*, 23(253):1–79, 2022.
- [18] X. Chen and L. Zhao. On the convergence of continuous single-timescale actor-critic. In *International Conference on Machine Learning*, 2025.
- [19] L. Da, J. Turnau, T. P. Kutralingam, A. Velasquez, P. Shakarian, and H. Wei. A survey of sim-to-real methods in RL: Progress, prospects and challenges with foundation models. *arXiv preprint arXiv:2502.13187*, 2025.
- [20] S. Dean, H. Mania, N. Matni, B. Recht, and S. Tu. Sample complexity of the linear quadratic regulator. *Foundations of Computational Mathematics*, 20:633–679, 2020.
- [21] J. Degraeve, F. Felici, J. Buchli, M. Neunert, B. Tracey, F. Carpanese, T. Ewalds, R. Hafner, A. Abdolmaleki, D. de Las Casas, et al. Magnetic control of Tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897):414–419, 2022.
- [22] F. Dörfler. Data-driven control: Part two of two: Hot take: Why not go with models? *IEEE Control Systems Magazine*, 43(6):27–31, 2023.
- [23] J. C. Doyle. Guaranteed Margins for LQG Regulators. *IEEE Transactions on Automatic Control*, 23(4):756–757, 1978.
- [24] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, et al. Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468, 2021.
- [25] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *International Conference on Learning Representations*, 2023.
- [26] O. Eberhard, M. Muehlebach, and C. Vernade. Partially observable reinforcement learning with memory traces. In *International Conference on Machine Learning*, volume 267, pages 14934–14949, 2025.
- [27] O. Eberhard, C. Vernade, and M. Muehlebach. Commit to the bit: Reactive reinforcement learning done right. In *International Conference on Machine Learning*, volume 306, 2026.
- [28] E. Elelimy, A. White, M. Bowling, and M. White. Real-time recurrent learning using trace units in reinforcement learning. *Advances in Neural Information Processing Systems*, 37:17006–17043, 2024.
- [29] G. Elmkaiel, S. Schmitt, and M. Muehlebach. Embodied intelligence for sustainable flight: A soaring robot with active morphological control. *npj Robotics*, 2026.
- [30] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pages 1587–1596, 2018.
- [31] D. Gabor, W. Wilby, and R. Woodcock. A universal non-linear filter, predictor and simulator which optimizes itself by a learning process. *Proceedings of the IEE-Part B: Electronic and Communication Engineering*, 108(40):422–435, 1961.
- [32] M. Gaur, A. S. Bedi, D. Wang, and V. Aggarwal. Closing the gap: Achieving global convergence (last iterate) of actor-critic under Markovian sampling with neural network parametrization. *arXiv preprint arXiv:2405.01843*, 2024.
- [33] N. Golowich, A. Moitra, and D. Rohatgi. Planning and learning in partially observable systems via filter stability. In *ACM Symposium on Theory of Computing*, pages 349–362, 2023.
- [34] G. Goodwin and K. Sin. *Adaptive Filtering Prediction and Control*. Prentice-Hall, 1984.
- [35] E. Graves, E. Imani, R. Kumaraswamy, and M. White. Off-policy actor-critic with emphatic weightings. *Journal of Machine Learning Research*, 24(146):1–63, 2023.
- [36] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018.
- [37] D. Hafner, W. Yan, and T. Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.

- [38] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- [39] Z. He, S. Bolognani, F. Dörfler, and M. Muehlebach. Decision-dependent stochastic optimization: The role of distribution dynamics. *arXiv preprint arXiv:2503.07324*, 2025.
- [40] L. Hewing, J. Kabzan, and M. N. Zeilinger. Cautious model predictive control using Gaussian process regression. *IEEE Transactions on Control Systems Technology*, 28(6):2736–2743, 2020.
- [41] K.-C. Hsu, H. Hu, and J. F. Fisac. The safety filter: A unified view of safety-critical control in autonomous systems. *Annual Review of Control, Robotics, and Autonomous Systems*, 7, 2023.
- [42] P. A. Ioannou and J. Sun. *Robust Adaptive Control*. Dover Publications, 2012.
- [43] T. Jaksch, R. Ortner, and P. Auer. Near-optimal regret bounds for reinforcement learning. *Journal of Machine Learning Research*, 11(51):1563–1600, 2010.
- [44] M. K. Janjua, H. Shah, M. White, E. Miah, M. C. Machado, and A. White. Gvfs in the real world: making predictions online for water treatment. *Machine Learning*, 113(8):5151–5181, 2024.
- [45] S. Kapturowski, G. Ostrovski, J. Quan, R. Munos, and W. Dabney. Recurrent experience replay in distributed reinforcement learning. In *International conference on learning representations*, 2018.
- [46] I. Karafyllis and M. Krstic. *Robust adaptive control: deadzone-adapted disturbance suppression*. SIAM, 2025.
- [47] P. Karmakar and S. Bhatnagar. Two time-scale stochastic approximation with controlled markov noise and off-policy temporal-difference learning. *Mathematics of Operations Research*, 43(1):130–151, 2018.
- [48] H. K. Khalil. *Nonlinear Systems*. Prentice Hall, 3rd edition, 2002.
- [49] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [50] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning*, pages 282—293, 2006.
- [51] A. Krishnamurthy, A. Agarwal, and J. Langford. PAC reinforcement learning with rich observations. *Advances in Neural Information Processing Systems*, 29, 2016.
- [52] M. Krstić, I. Kanellakopoulos, and P. V. Kokotović. *Nonlinear and Adaptive Control Design*. Wiley, 1995.
- [53] P. R. Kumar and A. Becker. A new family of optimal adaptive controllers for Markov chains. *IEEE Trans. on Automatic Control*, 27:137–146, 1982.
- [54] T. Lai and H. Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985. ISSN 0196-8858. doi: [https://doi.org/10.1016/0196-8858\(85\)90002-8](https://doi.org/10.1016/0196-8858(85)90002-8). URL <https://www.sciencedirect.com/science/article/pii/0196885885900028>.
- [55] T. Lattimore and C. Szepesvári. Sample-based planning and learning with function approximation. *Statistical Sciences*, 40(4):517–545, 2025.
- [56] M. Lauri, D. Hsu, and J. Pajarinen. Partially observable markov decision processes in robotics: A survey. *IEEE Transactions on Robotics*, 39(1):21–40, 2022.
- [57] N. P. Lawrence, T. Banker, and A. Mesbah. Mpcritic: A plug-and-play MPC architecture for reinforcement learning. In *IEEE Conference on Decision and Control*, pages 1048–1054, 2025.
- [58] F. L. Lewis and D. Vrabie. Reinforcement Learning and Adaptive Dynamic Programming for Feedback Control. *IEEE Circuits and Systems Magazine*, 9(3):32–50, 2009.
- [59] M. L. Littman. Memoryless policies: Theoretical limitations and practical results. In *International Conference on Simulation of Adaptive Behavior: From Animals to Animats*, volume 3, pages 238–245, 1994.
- [60] Q. Liu, A. Chung, C. Szepesvari, and C. Jin. When is partially observable reinforcement learning not scary? In *Conference on Learning Theory*, volume 178, pages 5175–5220, 2022.
- [61] T. Liu, Y. Song, L. Zhu, and D. J. Hill. Stability and control of power grids. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(1):689–716, 2022.
- [62] H. Ma, D. Büchler, B. Schölkopf, and M. Muehlebach. Reinforcement learning with model-based feedforward inputs for robotic table tennis. *Autonomous Robots*, 47(8):1387–1403, 2023.
- [63] G. Martius, R. Der, and N. Ay. Information driven self-organization of complex robotic behaviors. *PLOS One*, 8(5):e63400, 2013.
- [64] E. Mishkin and L. Braun. *Adaptive Control Systems*. McGraw-Hill Publishing, 1961.
- [65] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra,

- S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.
- [66] M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of Machine Learning*. MIT Press, second edition, 2018.
 - [67] M. Muehlebach, Z. He, and M. I. Jordan. The sample complexity of online reinforcement learning: A multi-model perspective. *International Conference on Learning Representations*, 2026.
 - [68] K. S. Narendra and A. M. Annaswamy. *Stable Adaptive Systems*. Prentice Hall, 1989.
 - [69] I. Osband and B. Van Roy. Why is posterior sampling better than optimism for reinforcement learning? In *International Conference on Machine Learning*, pages 2701–2710, 2017.
 - [70] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep exploration via bootstrapped DQN. *Advances in Neural Information Processing Systems*, 29, 2016.
 - [71] Y. Pan, M. Zaheer, A. White, A. Patterson, and M. White. Organizing experience: a deeper look at replay mechanisms for sample-based planning in continuous state domains. *International Joint Conference on Artificial Intelligence*, 2018.
 - [72] C. H. Papadimitriou and J. N. Tsitsiklis. The complexity of Markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.
 - [73] A. Patterson, A. White, and M. White. A generalized projected Bellman error for off-policy value estimation in reinforcement learning. *Journal of Machine Learning Research*, 23(145):1–61, 2022.
 - [74] A. Patterson, S. Neumann, M. White, and A. White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318):1–63, 2024.
 - [75] C. Piazza, G. Grioli, M. G. Catalano, and A. Bicchi. A century of robotic hands. *Annual Review of Control, Robotics, and Autonomous Systems*, 2(1):1–32, 2019.
 - [76] W. B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. John Wiley & Sons, 2007.
 - [77] F. Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386, 1958.
 - [78] S. Sastry and M. Bodson. *Adaptive control: stability, convergence and robustness*. Prentice Hall, 1989.
 - [79] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
 - [80] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning*, pages 387–395, 2014.
 - [81] D. e. a. Silver. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484—489, 2016.
 - [82] D. e. a. Silver. Mastering chess and Shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
 - [83] A. Smyshlyaev and M. Krstic. *Adaptive control of parabolic PDEs*. Princeton University Press, 2010.
 - [84] M. J. A. Strens. A Bayesian framework for reinforcement learning. In *ICML*, pages 943–950, 2000.
 - [85] B. Sukhija, M. Turchetta, D. Lindner, A. Krause, S. Trimpe, and D. Baumann. GoSafeOpt: Scalable Safe Exploration for Global Optimization of Dynamical Systems. *Artificial Intelligence*, 320, 2023.
 - [86] R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.
 - [87] R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. In *AAAI Conference on Artificial Intelligence*, pages 216–224, 1991.
 - [88] R. S. Sutton. The Bitter Lesson. Incomplete Ideas (Blog), 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
 - [89] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999.
 - [90] R. S. Sutton, A. G. Barto, and R. J. Williams. Reinforcement Learning is Direct Adaptive Optimal Control. *IEEE Control Systems Magazine*, 12(2):19–22, 2002.
 - [91] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone. Deep reinforcement learning for robotics: A survey of real-world successes. *Annual Review of Control, Robotics, and Autonomous Systems*, 8(1):153–188, 2025.

- [92] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. d. L. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- [93] M. Tayal, M. Tayal, A. Singh, S. Kolathaya, and R. Prakash. V-OCBF: Learning Safety Filters from Offline Data via Value-Guided Offline Control Barrier Functions. *Transactions on Machine Learning*, 2026.
- [94] P. Thomas. Bias in natural actor-critic algorithms. In *International Conference on Machine Learning*, pages 441–448, 2014.
- [95] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: a physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.
- [96] J. Tsitsiklis and B. Van Roy. Analysis of temporal-difference learning with function approximation. *Advances in Neural Information Processing Systems*, 9, 1996.
- [97] J. N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine learning*, 16(3):185–202, 1994.
- [98] H. P. Van Hasselt, M. Hessel, and J. Aslanides. When to use parametric models in reinforcement learning? *Advances in Neural Information Processing Systems*, 32, 2019.
- [99] Y. Wang, S. S. Zhan, R. Jiao, Z. Wang, W. Jin, Z. Yang, Z. Wang, C. Huang, and Q. Zhu. Enforcing Hard Constraints with Soft Barriers: Safe Reinforcement Learning in Unknown Stochastic Environments. In *International Conference on Machine Learning*, 2023.
- [100] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 5, 1992.
- [101] Y. Xie. A survey of safe reinforcement learning methods in robotics. *ITM Web of Conferences*, 2025.
- [102] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferrazza, Y. Tassa, and P. Abbeel. MuJoCo playground. *arXiv preprint arXiv:2502.08844*, 2025.
- [103] S. Zhang, B. Liu, H. Yao, and S. Whiteson. Provably convergent two-timescale off-policy actor-critic with function approximation. In *International Conference on Machine Learning*, pages 11204–11213, 2020.
- [104] Y. Zhao, O. Eberhard, M. Khammassi, A. H. Sayed, and M. Muehlebach. Why linear recurrent memory works in partially observable reinforcement learning. In *International Conference on Machine Learning*, volume 306, 2026.
- [105] Y. Zhu and M. Krstic. *Delay-adaptive linear control*. Princeton University Press, 2020.

A Notation and correspondence between RL and control

Throughout this tutorial we adopt a unified notation whenever possible. Since RL and control theory have historically developed different terminologies and conventions, Table 1 summarizes the main symbols used throughout the text and their correspondence with common notation in each community.

Table 1: Correspondence between the notation adopted in this tutorial and common notation in RL and control theory.

Concept	Tutorial	Common RL	Common Control
State	s_t	s_t	x_t
Action / control input	a_t	a_t	u_t
Policy / controller	π	π	$\kappa, u = \kappa(x)$
Parameterized policy	π_θ	π_θ	κ_θ
Dynamics	$f(s, a)$ or $p(\cdot s, a)$	P, p	f, A, B
Disturbance	η_t	often implicit	d_t, w_t, η_t
State space	$\mathcal{S}$	$\mathcal{S}$	$\mathcal{X}$
Action space	$\mathcal{A}$	$\mathcal{A}$	$\mathcal{U}$
Cost	$c(s, a)$	$-r(s, a)$	$\ell(x, u)$
Reward	$r(s, a)$	$r(s, a)$	rarely used
Objective (min.)	$J(\pi)$	uncommon	$J(\pi)$
Objective (max.)	$V(\pi)$	$J(\pi)$	uncommon
Discount factor	γ	γ	often omitted
Value function	V^π	V^π	cost-to-go
Optimal value function	V^*	V^*	optimal cost-to-go
Action-value function	Q^π	Q^π	state-action cost-to-go
Lyapunov function	rarely used	U	V
Advantage function	A^π	A^π	rarely used
State distribution	d^π	d^π	invariant measure
Actor	π_θ	π_θ	parameterized controller
Critic	q_w	Q_w, Q_ϕ	approximate cost-to-go
Reference signal	r_t	goal signal	r_t
Tracking error	e_t	uncommon	e_t
Feedback gain	K	uncommon	K